

Algorithmes fondamentaux du projet

Identification d'un réseau de drogue

DIMBIN'NY AINA Nomenjanahary Andrianina Geoffroy

Mai 2026

Table des matières

1	Introduction	2
2	Algorithme 1 : Nettoyage du graphe	2
2.1	Objectif	2
2.2	Fonction trouver_composantes(G)	2
2.3	Fonction nettoyage(aretes, N)	2
2.4	Propriétés du graphe nettoyé	3
3	Algorithme 2 : Classification du graphe	3
3.1	Coloration gloutonne	3
3.1.1	Fonction colorier_graphe(G)	3
3.2	Détection des cliques par Bron-Kerbosch	4
3.2.1	Fonction detecter_cliques(G, taille_min)	4
3.3	Interprétation criminologique des deux algorithmes	5

1 Introduction

Deux algorithmes fondamentaux structurent ce projet. Le premier assure le nettoyage du graphe brut. Le second réalise la classification des individus par coloration et détection de cliques.

2 Algorithme 1 : Nettoyage du graphe

2.1 Objectif

Transformer un graphe brut potentiellement sale (boucles, multi-arêtes, non connexe) en un graphe propre exploitable pour l'analyse criminologique.

2.2 Fonction trouver_composantes(G)

Cette fonction identifie toutes les composantes connexes du graphe.

Algorithm 1 Détection des composantes connexes

Require: G : un graphe avec un ensemble de sommets et d'arêtes

Ensure: Une liste d'ensembles contenant les sommets de chaque composante

```
visites  $\leftarrow \emptyset$ 
composantes  $\leftarrow []$ 
for chaque depart dans  $G.nodes()$  do
  if depart  $\in$  visites then
    continuer
  end if
  composante  $\leftarrow \emptyset$ 
  file  $\leftarrow [depart]$ 
  while file  $\neq []$  do
    sommet  $\leftarrow file.pop(0)$ 
    if sommet  $\in$  visites then
      continuer
    end if
    visites.add(sommet)
    composante.add(sommet)
    for chaque voisin dans  $G.neighbors(sommet)$  do
      if voisin  $\notin$  visites then
        file.append(voisin)
      end if
    end for
  end while
  composantes.append(composante)
end for
return composantes
```

2.3 Fonction nettoyage(aretes, N)

Cette fonction est le cœur de l'algorithme 1. Elle nettoie le graphe et le rend connexe.

Algorithm 2 Nettoyage du graphe

Require: *aretes* : liste des arêtes brutes, N : nombre de sommets

Ensure: Un graphe G propre et connexe

```
 $G \leftarrow$  nouveau graphe vide  
 $G.add\_nodes([0, 1, \dots, N - 1])$   
{Étape 1 : Suppression des boucles et des multi-arêtes}  
for chaque  $(u, v)$  dans aretes do  
    if  $u \neq v$  then  
         $G.add\_edge(u, v)$   
    end if  
end for  
{Étape 2 : Détection des composantes}  
 $composantes \leftarrow trouver\_composantes(G)$   
{Étape 3 : Rétablissement de la connexité}  
if  $len(composantes) > 1$  then  
     $noeud\_ref \leftarrow$  premier élément de  $composantes[0]$   
    for chaque composante dans  $composantes[1 :]$  do  
         $noeud\_comp \leftarrow$  premier élément de composante  
         $G.add\_edge(noeud\_ref, noeud\_comp)$   
    end for  
end if  
return  $G$ 
```

2.4 Propriétés du graphe nettoyé

Après exécution de l'algorithme 1, le graphe G' obtenu possède les propriétés suivantes :

- G' est connexe
- G' ne contient aucune boucle
- G' ne contient aucune arête multiple

3 Algorithme 2 : Classification du graphe

3.1 Coloration gloutonne

3.1.1 Fonction $colorier_graphe(G)$

Cette fonction attribue une couleur à chaque sommet en respectant la contrainte que deux sommets adjacents n'ont pas la même couleur.

Algorithm 3 Coloration gloutonne par degré décroissant

Require: G : un graphe propre

Ensure: Un dictionnaire associant à chaque sommet sa couleur

```
{Tri des sommets par degré décroissant}
sommets_tries  $\leftarrow$  tri( $G.nodes()$ , ordre décroissant du degré)
couleurs  $\leftarrow$  {}
for chaque sommet dans sommets_tries do
  {Collecte des couleurs des voisins déjà colorés}
  couleurs_voisins  $\leftarrow$   $\emptyset$ 
  for chaque voisin dans  $G.neighbors(sommet)$  do
    if voisin  $\in$  couleurs then
      couleurs_voisins.add(couleurs[voisin])
    end if
  end for
  {Recherche de la plus petite couleur disponible}
  couleur  $\leftarrow$  0
  while couleur  $\in$  couleurs_voisins do
    couleur  $\leftarrow$  couleur + 1
  end while
  couleurs[sommet]  $\leftarrow$  couleur
end for
return couleurs
```

3.2 Détection des cliques par Bron-Kerbosch

3.2.1 Fonction detecter_cliques(G , taille_min)

Cette fonction trouve toutes les cliques maximales de taille supérieure ou égale à un seuil donné.

Algorithm 4 Bron-Kerbosch : recherche des cliques maximales

Require: G : graphe, *taille_min* : taille minimale de clique à conserver

Ensure: *cliques* : liste des cliques trouvées, *appartenance* : dictionnaire

```
toutes_cliques  $\leftarrow$  []
BronKerbosch( $R, P, X$  { $R$  : clique en construction,  $P$  : candidats,  $X$  : exclus})
if  $P = \emptyset$  ET  $X = \emptyset$  then
  if  $\text{len}(R) \geq \text{taille\_min}$  then
    toutes_cliques.append(tri( $R$ ))
  end if
  return
end if
for chaque sommet dans une copie de  $P$  do
  voisins  $\leftarrow$  ensemble des voisins de sommet
  BronKerbosch( $R \cup \{sommet\}, P \cap voisins, X \cap voisins$ )
   $P \leftarrow P \setminus \{sommet\}$ 
   $X \leftarrow X \cup \{sommet\}$ 
end for
{Lancement initial}
BronKerbosch( $\emptyset$ , tous les sommets de  $G$ ,  $\emptyset$ )
{Tri des cliques par taille décroissante}
cliques  $\leftarrow$  tri(toutes_cliques, ordre décroissant de taille)
{Calcul de l'appartenance des sommets aux cliques}
appartenance  $\leftarrow$  { $n$  : [] pour  $n$  dans  $G.nodes()$ }
for chaque  $i, c$  dans énumérer(cliques) do
  for chaque  $n$  dans  $c$  do
    appartenance[ $n$ ].append( $i$ )
  end for
end for
return (cliques, appartenance)
```

3.3 Interprétation criminologique des deux algorithmes

Algorithme	Rôle algorithmique	Interprétation criminologique
Algorithme 1 : Nettoyage	Supprime les anomalies structurelles	Élimine les fausses relations et relie les groupes
Coloration gloutonne	Attribue des classes de sommets	Identifie les niveaux structurels équivalents
Bron-Kerbosch	Détecte les groupes où tous se connaissent	Identifie les cellules opérationnelles soudées

TABLE 1 – Synthèse des deux algorithmes fondamentaux